There are several parts to what the browser is sending. The first is the map itself. Inside a <script> tag is an object whose keys are the name of our modules exactly as we'd import them in our code, and the values are the URLs by which the browser can access those modules. Then there's a series of link tags that are of type modulepreload, which causes the browser to download the associated file and treat it as a JavaScript module. Toward the end, if you are on Safari, is the shim that makes import maps work on browsers that don't support them natively, and then a script statement to import application, our top-level entry point.

This works—see the code setup in Appendix 1, *Framework Swap* for more details.

At this point the config/importmap.rb file looks something like this:

```
pin "application", preload: true
pin "@hotwired/stimulus", to: "stimulus.min.js", preload: true
pin "@hotwired/stimulus-loading", to: "stimulus-loading.js", preload: true
pin_all_from "app/javascript/controllers", under: "controllers"
pin "@hotwired/turbo-rails", to: "turbo.min.js", preload: true
```

These pin statements are being used to drive what the import map sends to the browser. We see a few different types:

- The application doesn't specify a location, meaning that it is an entry point.
- The @hotwired entries all link to local files, which are part of the distribution when the turbo-rails or stimulus-rails gems are installed.
- The entire app/javascript/controllers directory comes in from that pin_all_from command.

The preload: true option covers what libraries get added to that list of link tags, meaning that the browser accesses them on page load and not on reference. This is something you might do for performance purposes so that the browser does not have to dig through a deep dependency tree in order to respond to a user click action or something.